Data Structures

Dilip Kumar Maripuri
Computer Applications

Data Structures

Session : Operations of Binary Trees - Create , Insert (Using Linked Lists)

Dilip Kumar Maripuri
Computer Applications

Operations on Binary Tree

- A **binary tree** is a hierarchical data structure where each node has at most **two children**: a **left child** and a **right child**.
- A binary tree consists of nodes, where:
  - Each node contains **data** (a value).
  - Each node has a **left** and a **right** pointer (pointing to child nodes).
  - The **root node** is the starting point of the tree.
  - Nodes are connected in a **hierarchical manner**.

Operations on Binary Tree

- **How to construct the Binary Tree**
  1. **If the tree is empty** (i.e., no root exists), the first node inserted becomes the **root** of the tree.
  2. For subsequent insertions, the algorithm follows a **level-order strategy**, meaning:
     - Nodes are filled **from top to bottom**.
     - Nodes are filled **from left to right** at each level before moving to the next level.
  3. A **queue-based approach** is used to keep track of nodes and find the first available position where a new node can be inserted.

Operations on Binary Tree

- **How Level-Order Insertion Works**
  - The tree is traversed level by level.
  - At each level:
    - If the **left child of a node** is empty, the new node is inserted there.
    - Otherwise, the left child is **added to the queue** for further checking.
    - If the **right child of a node** is empty, the new node is inserted there.
    - Otherwise, the right child is **added to the queue** for further checking.
  - This process ensures that all levels are filled **as evenly as possible**, making the tree **balanced**.

Define Tree Node

```
typedef struct Node {
    int data;
    struct Node* left;
    struct Node* right;
} *TreeNode;
```

Allocate Memory to the Node

```
TreeNode createNode(int value) {
    TreeNode newNode = (TreeNode)malloc(sizeof(
        struct Node));
    if (!newNode) {
        printf("Memory error!\n");
        return NULL;
    }
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}
```

Insert a node into the binary tree

```
TreeNode insert(TreeNode ROOT, int value) {
    TreeNode newNode = createNode(value);
    if (ROOT == NULL) {
        return newNode;
    }
    while (front < rear) {
        TreeNode temp = queue[front++];
        if (temp->left == NULL) {
            temp->left = newNode;
            return ROOT;        }
        else
            queue[rear++] = temp->left;
```

Insert a node into the binary tree

```
        if (temp->right == NULL) {
            temp->right = newNode;
            return ROOT;          }
        else
            queue[rear++] = temp->right;

    }
    return ROOT;
}
```

Main Function

```
int main() {
    TreeNode root = NULL;
    int values[] = {11,8,5,9,12,45,67,90};
    int n = sizeof(values) / sizeof(values[0]);

    for (int i = 0; i < n; i++) {
        root = insert(root, values[i]);

    // Display Tree
    // Can use DFS / BFS
    DFS(root);
    freeTree(root);
    }
```

```
void DFS(TreeNode root) {
    if (root == NULL) return;
    printf("%d ", root->data);
    DFS(root->left);
    DFS(root->right);
}
```

Thank You

Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu
8073212026